Figure 14.3 shows an example of a weighted graph with loops.

14.1.1 Implementing Graphs

The previous section formally defines graphs; however, when we move from theory to practice, often we have to face new issues and cope with constraints.

As much as mathematical notation makes it clear how we should represent graphs on paper, we need to decide, for instance, what's the best way to store them into a data structure.

There are several questions to answer, depending on the context: should we store labels for vertices and edges, or should we just assign an index to vertices using natural numbers, and enumerate edges following the natural ordering of pairs of indices?

While storing vertices is relatively easy (using lists, and possibly a dictionary to associate each vertex to its label), there also is another question that goes beyond any context: how should we store edges?

The question is not as trivial as it might look: the caveat is that at some point we will want to check if there is an edge between two vertices, or maybe find all outgoing edges of a certain vertex; if we just store all edges in a single list, either sorted or unsorted, then we will have to scan the whole list to find out our answers.